The advantage of using a single parameter vector is that it is easy to store all the parameters for multiple neural networks as a list of vectors. We just let the neural network unpack the vector into layer matrices. We use the `tanh` activation function because its output is in the interval [-1,1], and our reward is in the interval [-2,2], so a +2 reward will strongly push the Q value output toward +1. However, we want to be able to re-use this Q function for our later projects, so we provide the activation function as an optional keyword parameter, `afn`. In listing 9.6 we define some helper functions to produce state information from the environment (which is the grid).

Listing 9.6. The 1D Ising model: Get the state of the environment

```
def get_substate(b):                                                    1
    s = torch.zeros(2)
    if b > 0:                                                           2
        s[1] = 1
    else:
        s[0] = 1
    return s

def joint_state(s):                                                     3
    s1_ = get_substate(s[0])                                            4
    s2_ = get_substate(s[1])
    ret = (s1_.reshape(2,1) @ s2_.reshape(1,2)).flatten()               5
    return ret
```

- ***1*** Takes a single binary number and turns it into a one-hot encoded action vector like [0,1].
- ***2*** If the input is 0 (down), the action vector is [1,0]; otherwise it is [0,1].
- ***3*** s is a vector with 2 elements where s[0] = left neighbor, s[1] = right neighbor.
- ***4*** Gets the action vectors for each element in s
- ***5*** Creates the joint-action space using the outer-product, then flattens into a vector

The functions in listing 9.6 are two auxiliary functions we need to prepare the state information for the Q function. The `get_substate` function takes a single binary number (`0` for spin-down and `1` for spin-up) and turns it into a one-hot encoded action vector, where `0` becomes `[1,0]` and `1`